

Outils de qualité de code

Dispositif de qualité en place sur **Petit Rayon de Soleil** : analyse statique, formatage automatique, tests, conventions et revue. Chaque outil est réellement installé et intégré au pipeline d'intégration continue.

Projet Petit Rayon de Soleil Auteur de Breyne Hélió Outils ESLint · Prettier · node:test

1 Synthèse : les six pratiques

État réel de chaque pratique de qualité demandée, avec l'outil correspondant.

Pratique	Utilisé ?	Outil / moyen	Comment
ESLint	✓ oui	eslint 9	Analyse statique. <code>npm run lint</code> → 0 problème.
Prettier	✓ oui	prettier 3	Formatage automatique et uniforme. <code>npm run format</code> .
Tests automatisés	✓ oui	node:test	15 tests (unitaires + intégration). <code>npm test</code> .
Conventions de nommage	✓ oui	convention projet	Règles explicites et cohérentes (voir §3).
Organisation des commits	🕒 en cours	Conventional Commits	Convention adoptée pour la suite du projet (voir §4).
Revue manuelle du code	🕒 adapté	auto-revue + PR	Projet solo : relecture systématique + CI (voir §4).

2 ESLint & Prettier

Deux outils complémentaires, aux rôles bien séparés : ESLint traque les *erreurs*, Prettier impose un *style* uniforme. Une configuration (`eslint-config-prettier`) évite qu'ils se contredisent.

ESLint

ANALYSE

Détecte variables inutilisées, blocs vides, erreurs potentielles. Configuration adaptée au projet : contexte Node.js pour le serveur, contexte navigateur pour le dossier `public/` .

Prettier

FORMAT

Reformate automatiquement tout le code selon des règles fixes (guillemets simples, largeur 100, point-virgule). Fin des débats de style : le formatage n'est plus manuel.

Preuve d'exécution

bash – outils de qualité

```
$ npm run lint          # ESLint
(aucune sortie = 0 erreur, 0 avertissement)

$ npm run format:check  # Prettier
Checking formatting...
All matched files use Prettier code style!

$ npm test              # Tests
i tests 15 · pass 15 · fail 0
```

Intégrés à la CI. Ces trois contrôles (`lint`, `format:check`, `test`) sont rejoués automatiquement par le pipeline GitHub Actions à chaque push. Un code non conforme au style ou une erreur ESLint fait échouer le build avant tout déploiement.

3 Conventions de nommage

Un nommage cohérent rend le code lisible sans documentation. Les règles suivies dans le projet :

- **Variables & fonctions** : camelCase (`hashPassword` , `montrerVue` , `dernierMessageId`).
- **Constantes globales** : UPPER_CASE (`PORT` , `BASE` , `DB_FILES`).
- **Tables & colonnes SQL** : snake_case (`password_hash` , `propose_par` , `message_id`).
- **Routes REST** : chemins en minuscules, préfixe `/api/` , noms au pluriel (`/api/messages` , `/api/favoris`).
- **Fichiers** : minuscules, rôle explicite (`server.js` , `auth.js` , `reset-db.js` , `*.test.js`).
- **Argument non utilisé** : préfixé `_` (`_next`) pour signaler l'intention.

Français / anglais assumé. Le code métier visible (fonctions d'interface, messages) est en français, cohérent avec le domaine ; les termes techniques standards restent en anglais (token, hash, status). Ce choix est constant dans tout le projet.

4 Commits & revue de code

Organisation des commits

La convention retenue est **Conventional Commits** : chaque message commence par un type qui décrit la nature du changement.

exemples de messages de commit

```
feat:    ajout du statut 'rejected' pour les messages
fix:     correction du verrou de base lors des tests
test:    ajout du parcours de rejet en modération
chore:   mise en place d'ESLint et Prettier
docs:    annexe sur les environnements de déploiement
```

Types utilisés : `feat` (fonctionnalité), `fix` (correction), `test`, `docs`, `chore` (outillage), `refactor`. Cette convention rend l'historique lisible et faciliterait la génération automatique d'un changelog.

HONNÊTE L'historique initial du dépôt comporte quelques commits courts (« first commit », « capture »). La convention Conventional Commits est adoptée **à partir de maintenant** pour structurer la suite du travail — c'est une amélioration en cours, pas un acquis rétroactif.

Revue manuelle du code

Le projet étant développé en solo, il n'y a pas de relecture par un pair. La qualité est maintenue par un dispositif adapté :

- **Auto-revue** systématique avant chaque commit (relecture du diff).
- **Filet automatique** : ESLint + Prettier + 15 tests détectent ce qu'un œil humain manquerait.
- **Pull requests** : le workflow via PR est en place ; le pipeline CI joue le rôle de « relecteur automatique » qui doit passer au vert avant fusion.

En équipe, la suite naturelle serait d'exiger l'approbation d'un relecteur sur chaque PR (branch protection GitHub) — la mécanique de PR + CI est déjà prête à l'accueillir.

Petit Rayon de Soleil — Annexe « Outils de qualité de code ». États ESLint (0 problème), Prettier (conforme) et tests (15/15) vérifiés sur le projet. Configs : `eslint.config.js`, `.prettierrc.json`.